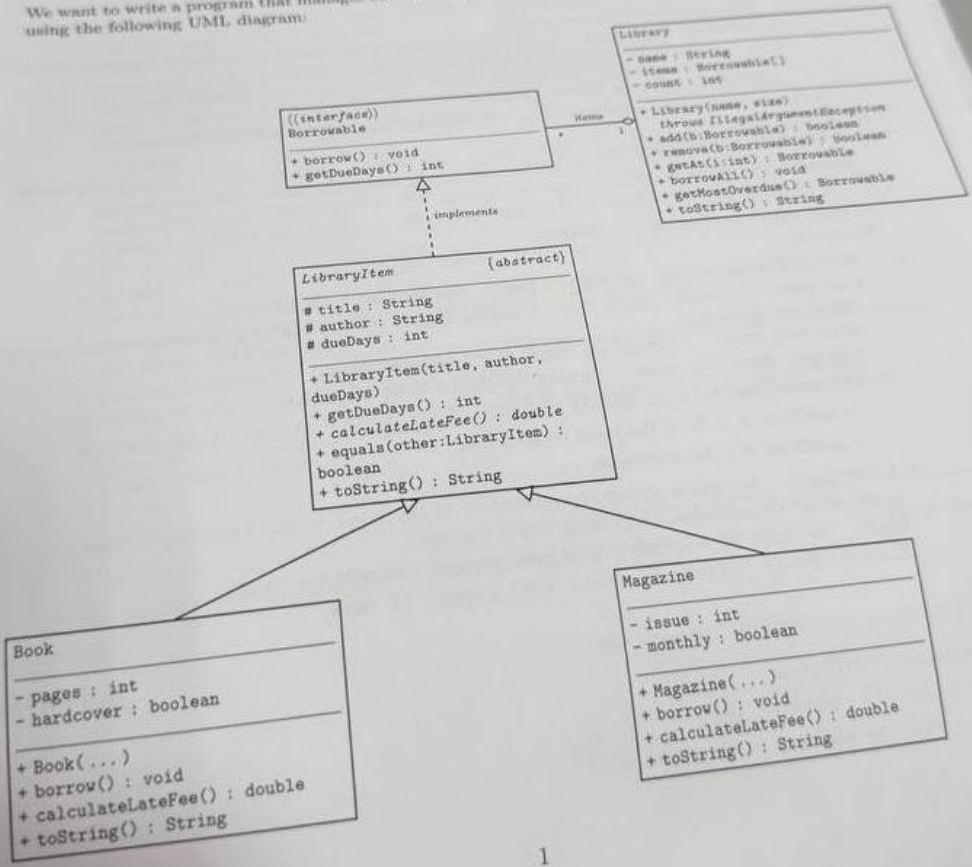
**Instructions**

- The use of any Java Collection or external library is **not allowed**.
- Open a new workspace before starting in your IDE.
- Write your name and ID at the top of every `.java` file as a comment.
- Submit all `.java` files to LMS. Any file not submitted receives 0.
- Duration: **1 hour 50 minutes**. Total marks: **10**.

**Scenario**

We want to write a program that manages borrowable items (books and magazines) inside a library, using the following UML diagram:



**Specification** [0.5 marks]

**Interface** `Borrowable`

**Methods (abstract):**

- `borrow()` : `void` : borrows the item.
- `getDueDays()` : `int` : returns the number of days the item has been overdue.

[1.5 marks]

**Abstract Class** `LibraryItem` (*implements* `Borrowable`)

**Instance Attributes:**

- `protected title` : `String`
- `protected author` : `String`
- `protected dueDays` : `int`

**Methods:**

- `LibraryItem(title, author, dueDays)` : constructor.
- `getDueDays()` : `int` : returns `dueDays`.
- `calculateLateFee()` : `double` : **abstract**.
- `equals(other` : `LibraryItem)` : `boolean` : two library items are equal if they have the same `title` and the same `author`.
- `toString()` : returns a string representation of the library item in the following format (`< ... >` marks an attribute value, everything else is literal):
  `<title> by <author> (<dueDays> days)`
  *Example:* `Gatsby by Scott (7 days)`

*Note:* `borrow()` *is not implemented in* `LibraryItem`*; each subclass provides its own.*

[1.25 marks]

**Class** `Book` ***extends*** `LibraryItem`

**Instance Attributes:**

- `private pages` : `int`
- `private hardcover` : `boolean`

**Methods:**

- `Book(title, author, dueDays, pages, hardcover)` : constructor.
- `borrow()` : `void` : prints in the following format (`< ... >` marks an attribute value, everything else is literal):
  `Borrowed book: <title> by <author> (<pages> pages)`
  *Example:* `Borrowed book: Gatsby by Scott (180 pages)`
- `calculateLateFee()` : `double` : returns:
  - `dueDays * 1.50` if the book is hardcover
  - `dueDays * 0.50` otherwise
- `toString()` : returns a string representation of the book in the following format (`< ... >` marks an attribute value, everything else is literal):
  `Book: <title> by <author> [<pages> pages] (<dueDays> days)`
  *Example:* `Book: Gatsby by Scott [180 pages] (7 days)`

[1.0 marks]

**Class** `Magazine` ***extends*** `LibraryItem`

**Instance Attributes:**

- `private issue` : `int`
- `private monthly` : `boolean`

**Methods:**

- `Magazine(title, author, dueDays, issue, monthly)` : constructor
- `borrow()` : `void` : prints in the following format (< . . . > marks an attribute value, everything else is literal):
  `Borrowed magazine: <title> Issue #<issue>`
  *Example:* `Borrowed magazine: NatGeo Issue #247`
- `calculateLateFee()` : `double` : returns `dueDays * 0.20`.
- `toString()` : returns a string representation of the magazine in the following format (< . . . > marks an attribute value, everything else is literal):
  `Magazine: <title> Issue #<issue> (<dueDays> days)`
  *Example:* `Magazine: NatGeo Issue #247 (3 days)`

**Class Library** **[3.75 marks]**

**Instance Attributes:**

- `private name : String`
- `private items : Borrowable[]`
- `private count : int`

**Methods:**

- `Library(name, size)` : constructor.
  If `size <= 0`, the constructor must explicitly throw:
  `throw new IllegalArgumentException("Invalid size: " + size);`
  (`IllegalArgumentException` is a Java built-in runtime exception.)
- `add(b : Borrowable) : boolean` : adds `b` to the first available space if there is space and `b` is not already in the library (use `equals` for comparison). Returns `true` if it succeeds, `false` otherwise.
- `remove(b : Borrowable) : boolean` : removes `b` from the library if it is there by **shifting all subsequent items one position to the left to preserve the original order**, then decrements `count`. Returns `true` if it succeeds, `false` otherwise.
  *Example:* if items are `[A, B, C, D]` and we remove `B`, the resulting array is `[A, C, D]` (**not** `[A, D, C]`).
- `getAt(i : int) : Borrowable` : returns the item at index `i` (index starts from 0), or `null` if `i` is out of range.
- `borrowAll() : void` : calls `borrow()` on every item in the library (polymorphism through the `Borrowable` interface).
- `getMostOverdue() : Borrowable` : **recursively** returns the item with the largest `dueDays` in the library, or `null` if the library is empty.
- `toString()` : returns a string representation of the library in the following format:

```
name Library (count):
item1
item2
...
```

*Note: `getMostOverdue()` must be implemented recursively. An iterative solution will receive 0 marks.*

**Q1 : Implementation** [≈ 8.0 marks total]

Write the interface `Borrowable` and the classes `LibraryItem`, `Book`, `Magazine`, and `Library` as specified above.

**Q2 : `LibraryTest` (main method)** [2.0 marks]

Write a class `LibraryTest` containing a `main` method that demonstrates the functionality of your work.

The `main` method should:

1. First, attempt to create a `Library` with an **invalid size** (e.g. 0) inside a `try / catch` block, catching `IllegalArgumentException` and printing its message.
2. Then create a valid `Library` named `"Central"` with size 4.
3. Create the following items and add them to the library:
   - `Book("Gatsby", "Scott", 7, 180, false)`
   - `Book("Java", "Eckel", 14, 1000, true)`
   - `Magazine("NatGeo", "NGS", 3, 247, true)`
   - `Magazine("Time", "TI", 5, 1850, false)`
4. Print the library.
5. Call `borrowAll()` on the library.
6. Print the **most overdue** item in the library (using `getMostOverdue()`).

**Sample run**

```
Invalid size: 0
-----------------------------------------
Central Library (4):
Book: Gatsby by Scott [180 pages] (7 days)
Book: Java by Eckel [1000 pages] (14 days)
Magazine: NatGeo Issue #247 (3 days)
Magazine: Time Issue #1850 (5 days)
-----------------------------------------
Borrowed book: Gatsby by Scott (180 pages)
Borrowed book: Java by Eckel (1000 pages)
Borrowed magazine: NatGeo Issue #247
Borrowed magazine: Time Issue #1850
-----------------------------------------
Most overdue: Book: Java by Eckel [1000 pages] (14 days)
```